



# CP Bootcamp 2026 — Day 10

## Theme: Recursion + Backtracking Foundation

### Main Goal:

আজকে তুমি এমন একটা thinking pattern শিখবে যেখানে একটি function নিজেকেই call করে।

আগে:

Loop

↓

Repeat Work

আজ:

Function

↓

Call Itself

↓

Solve Smaller Version

↓

Build Answer

**Estimated Time:** 6–7 hours



# Day 10 Chapter Map

Day 10

- Chapter 1 – Recursion Mental Model
- Chapter 2 – Base Case & Recursive Case
- Chapter 3 – Recursion Trace & Call Stack
- Chapter 4 – Basic Recursion Problems
- Chapter 5 – Recursion on Arrays
- Chapter 6 – Recursion on Strings
- Chapter 7 – Backtracking Mental Model
- Chapter 8 – Generate All Possibilities
- Chapter 9 – Common Bugs & Edge Cases
- Chapter 10 – Day 10 Assignment & Practice

## Chapter 1 — Recursion Mental Model

### Recursion কী?

Recursion হলো:

একটি function নিজেকেই call করে।

Example:

```
void hello()  
{  
    hello();  
}
```

এটা infinite চলবে।

তাই recursion-এর দুইটা অংশ লাগে:

```
Function  
├── Base Case  
└── Recursive Case
```

## Chapter 2 — Base Case & Recursive Case

### Base Case

যেখানে recursion থামবে।

Example:

```
Print 1 to 5
```

Thinking:

```
print(1)
```

```
print(2)
```

```
print(3)
```

```
print(4)
```

```
print(5)
```

```
STOP
```

Stop condition:

```
n == 0
```

## Recursive Case

যেখানে function আবার call করে।

Example:

```
print(n-1);
```

Complete:

```
void printNumber(int n)
{
    if(n==0)
        return;

    printNumber(n-1);

    printf("%d\n",n);
}
```

Call:

```
printNumber(5);
```

Flow:

`print(5)`

↓

`print(4)`

↓

`print(3)`

↓

`print(2)`

↓

`print(1)`

↓

`stop`

## Chapter 3 — Recursion Trace & Call Stack

Recursion বুঝতে হলে stack বুঝতে হবে।

Example:

`factorial(4)`

Meaning:

```
[  
4! = 4 \times 3 \times 2 \times 1  
]
```

Function:

```
int factorial(int n)  
{  
    if(n==1)  
        return 1;  
  
    return n * factorial(n-1);  
}
```

Call:

```
factorial(4)
```

Stack:

```
factorial(4)
```

```
4 * factorial(3)
```

```
factorial(3)
```

```
3 * factorial(2)
```

```
factorial(2)
```

```
2 * factorial(1)
```

```
factorial(1)
```

```
return 1
```

Return:

```
1
```

```
2*1 = 2
```

```
3*2 = 6
```

```
4*6 = 24
```

Answer:

```
24
```

# Mental Model

Recursion:

Go Down

↓

Reach Base Case

↓

Come Back Up

## Chapter 4 — Basic Recursion Problems

### Problem 1 — Print 1 to N

Example:

Input:

5

Output:

1 2 3 4 5

Code:

```
void print(int n)
{
    if(n==0)
        return;

    print(n-1);

    printf("%d ",n);
}
```

Pattern:

Decrease Problem

↓

Solve Smaller

↓

Print While Returning

## Problem 2 — Print N to 1

Output:

5 4 3 2 1

Code:

```
void print(int n)
{
    if(n==0)
        return;

    printf("%d ",n);

    print(n-1);
}
```

Difference:

Before recursive call:

N to 1

After recursive call:

1 to N

Important recursion concept!

## Problem 3 — Sum 1 to N

Formula:

[  
1+2+3+...+n  
]

Recursive idea:

sum(n)

=

n + sum(n-1)

Code:

```
int sum(int n)
{
    if(n==0)
        return 0;

    return n + sum(n-1);
}
```

Example:

sum(5)

=5+sum(4)

=5+4+sum(3)

...

=15

## Problem 4 — Factorial

Already seen:

```
int fact(int n)
{
    if(n==1)
        return 1;

    return n*fact(n-1);
}
```

## Chapter 5 — Recursion on Arrays

Array:

5 3 8 1

### Print Array Recursively

```
void printArray(int arr[], int index, int n)
{
    if(index==n)
        return;

    printf("%d ",arr[index]);

    printArray(arr,index+1,n);
}
```

Flow:

index 0

↓

index 1

↓

index 2

↓

index 3

↓

stop

# Find Maximum Recursively

Idea:

Maximum of whole array

=

Current element

+

Maximum of rest

Code:

```
int maxArray(int arr[], int index, int n)
{
    if(index==n-1)
        return arr[index];

    int rest=maxArray(arr,index+1,n);

    if(arr[index]>rest)
        return arr[index];

    return rest;
}
```

## Chapter 6 — Recursion on Strings

### Reverse String

String:

hello

Need:

olleh

Two pointer recursion:

```
void reverse(char str[], int left, int right)
{
    if(left >= right)
        return;

    char temp=str[left];

    str[left]=str[right];

    str[right]=temp;

    reverse(str, left+1, right-1);
}
```

Pattern:

Two Pointer

+

Recursion

# Palindrome Check Recursively

Example:

madam

Compare:

m = m

a = a

d = d

Code:

```
int palindrome(char str[], int left, int right)
{
    if(left >= right)
        return 1;

    if(str[left] != str[right])
        return 0;

    return palindrome(str, left+1, right-1);
}
```

## Chapter 7 — Backtracking Mental Model

Recursion:

One path

Backtracking:

Many paths

Example:

Choices:

A

B

C

Need:

সব possible arrangement।

Backtracking:

Choose

↓

Explore

↓

Undo Choice

↓

Try Another

Diagram:

Start

/ | \

A B C

/ | \

...

## Chapter 8 — Generate All Possibilities

## Subsequence Problem

String:

abc

Possible:

""

"a"

"b"

"c"

"ab"

"ac"

"bc"

"abc"

At every character:

Two choices:

Take

or

Don't Take

Decision Tree:

```
      a
    /  \
  take  skip
  a      ""
```

Recursive Idea:

```
solve(index, current)
```

Pseudo:

```
if index == length
```

```
    print answer
```

```
else
```

```
    take character
```

```
    solve(next)
```

```
    remove character
```

```
    solve(next)
```

# Backtracking Pattern

Choose

↓

Recursive Call

↓

Undo Choice

## Chapter 9 — Common Bugs & Edge Cases

### Bug 1 — No Base Case

Wrong:

```
void fun(int n)
{
    fun(n-1);
}
```

Result:

Stack Overflow

## Bug 2 — Wrong Base Case

Example:

```
if(n==0)
```

কিন্তু function:

n-1

করছে।

Need careful।

## Bug 3 — Forget Undo in Backtracking

Wrong:

Choose

↓

Call

কিন্তু:

Undo

নেই।

Result:

Wrong answers।

## Bug 4 — Too Deep Recursion

Large:

```
n = 1000000
```

Recursion:

Stack overflow হতে পারে।

## Chapter 10 — Day 10 Assignment

### Part A — Basic Recursion

Implement:

1. Print 1 to N
2. Print N to 1
3. Sum 1 to N
4. Factorial
5. Fibonacci

### Part B — Array Recursion

Implement:

1. Print array
2. Sum array
3. Maximum element
4. Minimum element
5. Linear search recursively

## Part C — String Recursion

Implement:

1. Reverse string
2. Check palindrome
3. Count characters

## Part D — Backtracking Practice

Generate:

1. All subsequences of string
2. All binary strings of length N

Example:

N = 3

Output:

```
000
001
010
011
100
101
110
111
```

## Part E — Dry Run

Trace:

```
factorial(4)
```

Write:

Call:

Return:

Final Answer:

## Glossary Update

Add:

Recursion

Base Case

Recursive Case

Call Stack

Stack Overflow

Backtracking

Decision Tree

Choice

Explore

Undo Choice

State

Recursion Depth

# Pattern Library Update

Add:

## ## Recursion Pattern

Recognition:

Problem can be reduced into smaller same problems.

Mental Model:

Solve(n)

↓

Solve(n-1)

↓

Combine Answer

Need:

Base Case

Recursive Case

## ## Backtracking Pattern

Recognition:

Need all possible answers.

Mental Model:

Choose

↓

Explore

↓

Undo

Examples:

- Subsequence
- Permutation
- Combination
- Maze

# Day 10 Completion Checklist

- ☐ Recursion concept বুঝি
- ☐ Base case বুঝি
- ☐ Recursive case বুঝি
- ☐ Call stack বুঝি
- ☐ Factorial solve করতে পারি
- ☐ Sum recursion solve করতে পারি
- ☐ Array recursion বুঝি
- ☐ String recursion বুঝি
- ☐ Palindrome recursion বুঝি
- ☐ Backtracking idea বুঝি
- ☐ Choice → Explore → Undo pattern বুঝি
- ☐ Decision tree বুঝি



# Day 10 Final Mental Model

Big Problem



Make Smaller Same Problem



Recursive Call



Reach Base Case



Return Answer

আর Backtracking:

Problem

↓

Make Choice

↓

Explore

↓

Undo

↓

Try Next Choice

Day 10 শেষে তোমার CP foundation:

Day 1-5

Pattern Thinking

+

Day 6

Functions

+

Day 7

Searching

+

Day 8

Sorting + Complexity

+

Day 9

Optimization Patterns

+

Day 10

Recursion + Backtracking

পরের **Day 11 — Stack, Queue + Data Structure Thinking** হবে।